A2: Software Testing Assignment

MUST do it with your ***existing project group***

This assignment aims to improve the tests for a supplied room-booking program. Check what your tests cover, see whether they catch code changes, and repair one bug.

Deadline: November 16, 2026. Work with the same members as your course project group. Parts 1–6 share 93 marks. Quality is 2 marks. Part 7 gives each member up to 5 marks. **Each group should make one submission by the deadline.**

Start here

Write all code and automated tests in Java 17, including extensions and bug fixes. Use JUnit for core tests. Follow booking-tests/README.md for setup. Run the starter tests and save your work in Git. Keep reference code and tool settings unchanged. Use separate branches for repairs and manual mutations.

Summary of the requirement: We have created *Rules* that your tests must check

R1 Valid times. A booking starts after now and ends after its start. Check this before checking conflicts. Invalid times return INVALID_TIME and save nothing.

R2 Room conflicts. ACTIVE bookings cannot overlap in the same room. A conflict returns CONFLICT. One booking may start exactly when another ends. Different rooms and inactive bookings do not conflict. Success returns CREATED and saves one ACTIVE booking.

R3 Permission and state. Check ownership first. A non-owner receives DENIED. For an owner, a CANCELLED or COMPLETED booking returns INACTIVE.

R4 Cancellation time. An owner can cancel an ACTIVE booking at least 120 minutes before its start. Exactly 120 is allowed. Less than 120 returns TOO_LATE.

R5 Saved data and notifications. Successful cancellation returns CANCELLED, saves that state, and requests one notification. Rejected requests change nothing and request no notification. Cancelling again adds no request. Creating a booking sends no notification. Leave other bookings unchanged.

Test one request at a time with valid IDs, in-memory storage, and UTC times rounded to whole minutes. Use the supplied clock and sample states. You do not need to test null inputs, login, concurrent requests, real databases, email delivery, or implement completion. Assume dependencies succeed except in Part 6 Option B.

1 Plan your tests 15 marks

Save your initial plan in Git before reading method bodies. Then inspect the code and explain important plan changes. Include input groups and boundary values, such as 119, 120, and 121 minutes, plus touching, overlapping, and separate bookings.

Make a cancellation decision table using owner, state, time remaining, and expected result. Plan two sequences: cancel successfully then cancel again, and try as a non-owner then as the owner.

Use the test-map template to link R1–R5 to test names. Record starting data, actions, expected response, saved data, notifications, and why each case matters. Derive expected answers from the rules, not from the method being tested.

Marks: input groups and boundaries 5, decision table and sequences 5, test map and expected results 5.

Write and evaluate your tests

2 Write automated tests 18 marks

Write JUnit tests for successful requests, rejections, boundaries, and both sequences. Check responses, saved data, and notification requests. Use the recording notifier to inspect requests. Explain one bug that a return-value-only check could miss.

Give each test fresh data and a fixed clock. Avoid sleeps, live services, test-order dependencies, and the real clock. Use parameterized tests where useful. There is no required test count. Your final tests must pass on the reference version.

Marks: behaviour and boundaries 8, useful assertions 6, independent and readable tests 4.

3 Check coverage 15 marks

Run JaCoCo on the starter tests and your final tests. Save both reports, covered and total lines and branches, commands, and Git commit IDs. Coverage measures code executed, not whether results were checked. Measure BookingService only, as configured.

Choose two uncovered cases in the starter report. Add tests and explain the rule and expected result each checks. For one small method, list its decision outcomes and the tests that execute them. Compare this with JaCoCo and explain one remaining weakness.

Aim for 90% line and 85% branch coverage. Explain any shortfall. These are investigation targets, not automatic grade cutoffs. JaCoCo does not count exception handling as branches.

Marks: before and after reports 4, two gap investigations 6, reasoning and limits 5.

4 Investigate three mutations 20 marks

A mutant is a small code change. It is killed when a test detects the change, survives when tests still pass, or has NO_COVERAGE when no test reaches it. Finish your rule-based tests and commit them, run PIT, improve tests where needed, then run PIT again. Save both reports. A starter PIT run is not required.

Analyze three mutants. Include changing minutes < 120 to minutes <= 120. If PIT does not generate it, try it on a temporary branch, restore the code, and report it separately from PIT totals. For the other two, prefer surviving or uncovered mutants. Use killed mutants if too few remain.

For each, record the location, change, tool status, related rule, and why a test detects or misses it. Add a test or improve an assertion when needed, then record the rerun. If already detected, identify the assertion. Existing strong tests receive credit.

Report the score, its calculation, and counts by status. Keep timeouts and tool errors separate from assertion-based detection. An equivalent mutant changes no allowed behaviour. Justify that claim across allowed inputs. Do not change PIT targets to raise the score. No mutation percentage is required. Explain what mutation testing reveals beyond coverage.

Marks: repeatable runs 4, three analyses 9, improvements and explanation 7.

5 Find and fix one bug 15 marks

After writing your tests, apply the supplied patch using faulty-version/README.md. Show the same test passing on the reference, failing an assertion on the faulty version, and passing after repair. Run all tests on the repair and save the output. Keep repairs separate from the reference used for coverage and PIT.

Record the broken rule, reproduction steps, expected and actual results, cause, small fix, and run evidence. Explain one risk that rerunning the suite does not fully check.

Marks: detecting test 6, explanation and fix 5, regression checks 4.

Choose one extension and submit

6 Learn one technique 10 marks

Choose A or B. You only need one option.

Option A: Generate inputs. Use the supplied property-testing setup to check that a non-owner cannot change a booking by trying to cancel it. Generate different owners, callers, and states. Check rejection and unchanged storage. Run at least 100 cases and save the random seed so you can repeat the run. Explain your generated inputs and shrinking, which reduces a failing case to a simpler one. You do not need to introduce another bug.

Option B: Test a failed save. Use a controllable replacement repository. Make it throw RepositoryException before saving an otherwise valid cancellation. Check that the caller receives the exception, stored data stays unchanged, and no notification is requested. Explain why an always-successful replacement misses this case and one thing your replacement cannot prove.

Write a learning record of at most 250 words. Name the documentation you used, describe your first attempt and changes, and explain what you learned. Include it in the shared report.

Marks: working experiment 5, learning record 3, explanation and limits 2.

7 Explain your work individually 5 marks

Each student must contribute to the tests. Giving a student only document writing, or another trivial task, does not count. Each student submits up to 250 words (one PDF page). Name one boundary test you wrote and explain its expected result. Explain one analyzed mutant and the assertion that detects it or is missing. Name one remaining weakness and a useful next test. Refer to tests you wrote.

Marks: boundary and expected result 2, mutation explanation 2, limitation 1.

One group submission by November 16, Monday

Submit one ZIP per project group: SENG300_Testing_Group_<group-number>.zip. Include all names, student IDs, and the group number in your README. Submit the Java source project and report content for every part as follows.

Code: include one runnable Java project with source files, tests, the chosen extension, and a separate bug repair branch or patch. Include exact run commands. Cite code paths and test names in each report section. Do not submit only screenshots or compiled classes.

Report: use one shared report with sections Part 1–Part 6, at most three pages total including references and the learning record. Keep detailed records and tool reports separate from this limit. Include one named Part 7 file per member, up to 250 words each. Follow the PDF format in booking-tests/README.md.

Part 1: planning records, decision table, sequences, and test map. No separate code.
Part 2: Java tests, assertion choices, and test results.
Part 3: Java tests for two gaps, coverage comparison, and both JaCoCo reports.
Part 4: Java tests and assertions for three mutants, analysis, and both PIT reports.
Part 5: Java regression test, repaired source or patch, bug record, and run evidence.
Part 6: Java extension code, results, limits, and learning record up to 250 words.
Part 7: each member's explanation of tests they wrote. Each member must contribute to the tests. No additional code.

Include relevant Git commit IDs, raw tool results, and the contribution record for all members. Use the supplied templates. Save evidence before running clean, which deletes target/.

Quality 2 marks

This is judged across the whole group submission. It is left up to the judgement of the grading TA to award to high-quality submissions.

How marks work

Your mark is the group score out of 95, including the 2 quality marks, plus your individual score out of 5. Marks reward useful tests and clear reasoning. Similar to A1, we will follow the policies for collaboration, use of AI, and late policies. Acknowledge assistance as required.